

% Curso de Programacion con DragonScript
% Aprender a programar desde cero, al estilo Dragon Ball

Introduccion

DragonScript es un lenguaje de programacion real (interpretado, escrito en Python) pensado para **aprender a programar desde cero**. Todas sus palabras clave son del universo Dragon Ball y estan **en espanol**, para que puedas concentrarte en las ideas y no en el ingles.

Este curso sigue la secuencia pedagogica del enfoque **Gobstones** (el que se usa en Introduccion a la Programacion de la UNQ): cada leccion agrega **una sola idea nueva** sobre las anteriores. Si las haces en orden, no necesitas saber nada de programacion para empezar.

Las 16 lecciones estan en la carpeta `examples/curso/`. Cada archivo `.ds` esta lleno de comentarios explicativos: **este documento es el acompanamiento** que resume la teoria de cada una.

Como ejecutar una leccion (Windows)

Abri una terminal en la carpeta del proyecto y escribi:

```
python main.py examples\curso\01_programas.ds
```

Cambia el nombre del archivo para correr otra leccion. Tambien puedes abrir el proyecto en **Visual Studio Code** y apretar **F5** (ver `EJECUTAR_F5.pdf`).

Leccion 01 - Programas

Un **programa** es una secuencia de instrucciones que la computadora ejecuta de arriba hacia abajo, una tras otra. El orden importa.

La instruccion mas basica es **SCOUTER**, que muestra (imprime) un valor en pantalla, como el rastreador de los saiyajin.

```
SCOUTER "Bienvenido al planeta Vegeta"  
SCOUTER 9000 + 1
```

Las lineas que empiezan con `#` son **comentarios**: la computadora los ignora, sirven para dejar notas a los humanos.

Leccion 02 - Procedimientos y contratos

Un **procedimiento** es un bloque de instrucciones con un **nombre**, para usarlo (invocarlo) cuantas veces quieras sin copiar y pegar. Se define con

TECNICA .

```
TECNICA saludarGoku() {  
    SCOUTER "Hola, soy Goku!"  
}  
  
saludarGoku()    # lo invocamos
```

El **contrato** es el comentario que va antes de la tecnica y explica **QUE** hace (su proposito), sin importar **COMO** lo hace. Es una promesa para quien la use.

Leccion 03 - Repeticion simple

Cuando sabemos la cantidad **exacta** de repeticiones, usamos **GRAVEDAD** (como la camara de gravedad donde se repite el mismo ejercicio):

```
GRAVEDAD 5 {  
    SCOUTER "Flexion completada!"  
}
```

Eso muestra el mensaje 5 veces.

Leccion 04 - Parametros

Un procedimiento se vuelve mucho mas util si recibe **datos** que cambian su comportamiento. Esos datos se llaman **parametros** y van entre parentesis:

```
TECNICA presentar(nombre) {  
    SCOUTER "Guerrero en escena: " + nombre  
}  
  
presentar("Goku")    # "Goku" es el argumento  
presentar("Vegeta")
```

El mismo procedimiento sirve para muchos casos distintos.

Leccion 05 - Expresiones y tipos

Una **expresion** es algo que la computadora evalua para obtener un **valor**.
Cada valor tiene un **tipo**:

- **Numeros:** 9000 , 3 , 2.5
- **Textos:** "Kamehameha" (siempre entre comillas)
- **Booleanos:** CANON (verdadero) y RELLENO (falso)
- **Nulo:** VACIO (ausencia de valor)

Operadores aritmeticos: + - * / % . El + tambien une textos (concatenacion). Comparaciones: > < >= <= == != . Operadores logicos: **FUSION** (y), **DESEO** (o), **INVERTIR** (no).

```
SCOUTER 6 * 7           # 42
SCOUTER "Son" + "Goku"  # SonGoku
SCOUTER 9000 > 8000     # TRUE
SCOUTER CANON DESEO RELLENO # TRUE
```

Leccion 06 - Alternativa condicional

El programa **decide** entre caminos segun una condicion, con **SENSAR** (si) y **ESQUIVAR** (si no). Se pueden encadenar con **ESQUIVAR SENSAR** .

```
SENSAR poder > 9000 {
    SCOUTER "Supera los 9000!"
} ESQUIVAR SENSAR poder > 1000 {
    SCOUTER "Guerrero de elite"
} ESQUIVAR {
    SCOUTER "Necesita entrenar"
}
```

Leccion 07 - Funciones simples

Un procedimiento **hace** cosas; una **funcion calcula y devuelve** un valor con **TRANSMITIR** . Apenas se ejecuta **TRANSMITIR** , la tecnica termina y entrega ese valor.

```
TECNICA doble(n) {
    TRANSMITIR n * 2
}

KI resultado = doble(21)      # resultado vale 42
SCOUTER doble(doble(10))     # se pueden combinar
```

Leccion 08 - Repeticion condicional

Cuando **no** sabemos cuantas veces repetir, repetimos **mientras** se cumpla una condicion, con **ENTRENAR** (seguir entrenando mientras haga falta):

```
KI cuenta = 3
ENTRENAR cuenta > 0 {
    SCOUTER cuenta
    cuenta -= 1
}
```

Cuidado: dentro del ciclo algo debe cambiar para que la condicion eventualmente sea falsa; si no, el ciclo nunca termina.

Leccion 09 - Variables

Una **variable** es un espacio con nombre donde guardamos un valor que podemos leer y **cambiar**. Se declara con **KI** :

```
KI poder = 9000      # declarar
poder = 9001         # cambiar
poder += 500         # atajo: poder = poder + 500
```

Un **acumulador** es una variable que va sumando dentro de un ciclo.

Leccion 10 - Funciones con procesamiento

Funciones que, para calcular su resultado, hacen un **proceso** interno (variables locales, condicionales, ciclos) antes de **TRANSMITIR** .

```
TECNICA factorial(n) {
    KI resultado = 1
    KI i = 1
    ENTRENAR i <= n {
        resultado *= i
        i += 1
    }
    TRANSMITIR resultado
}
```

Leccion 11 - Recorrido de acumulacion

Un **recorrido** visita, uno por uno, todos los elementos de un grupo, con **RASTREAR ... EN ...**. Cuando ademas vamos sumando/combinando en una variable, es un **recorrido de acumulacion**:

```
KI total = 0
RASTREAR p EN [500, 1200, 9000] {
    total += p
}
SCOUTER total      # 10700
```

Con el mismo patron se calcula el maximo o se cuentan elementos que cumplen algo.

Leccion 12 - Busqueda

Buscar es recorrer un grupo para encontrar un elemento que cumple una condicion. Dos situaciones:

- **Sabiendo que esta:** tenemos certeza de que existe; apenas lo hallamos, lo guardamos.
- **Sin saber si esta:** puede no existir, asi que usamos un booleano **encontrado** que arranca en **RELLENO** y pasa a **CANON** solo si aparece.

```
TECNICA estaPresente(lista, nombre) {
    KI encontrado = RELLENO
    RASTREAR x EN lista {
        SENSAR x == nombre {
            encontrado = CANON
        }
    }
    TRANSMITIR encontrado
}
```

Leccion 13 - Recorridos sobre enumerativos (rangos)

Para recorrer numeros consecutivos usamos un **rango** **[inicio..fin]**, que genera la lista de enteros de inicio a fin (ambos incluidos):

```
SCOUTER [1..5]      # [1, 2, 3, 4, 5]

RASTREAR n EN [1..10] {
    SCOUTER "7 x " + n + " = " + (7 * n)
}
```

Tambien existe la funcion equivalente `RANGO(inicio, fin)`.

Leccion 14 - Registros y variantes

Un **registro** agrupa varios datos relacionados bajo una sola entidad. En vez de tener sueltos `nombre`, `raza` y `poder`, los juntamos en un **CAPSULA** con esos **campos**.

- Definir: `CAPSULA Peleador { ... }`
- Crear (**ACTIVAR**): `ACTIVAR Peleador("Goku", "Saiyajin", 9000)`
- Acceder a un campo: `objeto.campo`
- Modificar un campo (dentro de una tecnica): `Y0.campo = valor`

`Y0` se refiere al propio registro. Las **variantes** (distintos tipos del mismo concepto) se pueden modelar con un campo, por ejemplo `raza`.

```
CAPSULA Peleador {
    TECNICA __init__(nombre, raza, poder) {
        Y0.nombre = nombre
        Y0.raza = raza
        Y0.poder = poder
    }
    TECNICA ficha() {
        TRANSMITIR Y0.nombre + " [" + Y0.raza + "] poder=" + Y0.poder
    }
}

KI goku = ACTIVAR Peleador("Goku", "Saiyajin", 9000)
SCOUTER goku.ficha()
SCOUTER goku.nombre
```

Leccion 15 - Listas

Una **lista** (o grupo) es una coleccion **ordenada** de valores, entre corchetes y separados por comas. Cada elemento tiene una **posicion** (indice) que empieza en **0**.

```
KI equipo = ["Goku", "Vegeta", "Gohan"]
SCOUTER equipo[0] # Goku
equipo[0] = "Goku (SSJ)" # modificar por indice
```

Funciones para manipular listas:

Funcion	Que hace
LONGITUD(lista)	cuantos elementos tiene
ESTA_VACIA(lista)	CANON si no tiene elementos
CABEZA(lista)	el primer elemento
COLA(lista)	todos menos el primero
ABSORBER(lista, x)	una lista nueva con x al final
lista1 + lista2	une (concatena) dos listas

Leccion 16 - Recorridos y procesamiento de listas

Combinamos todo: recorreremos listas con `RASTREAR` y las procesamos para producir nuevos resultados. Tres patrones clasicos:

- **Transformar:** crear una lista nueva aplicando algo a cada elemento.
- **Filtrar:** crear una lista nueva solo con los que cumplen una condicion.
- **Reducir:** combinar todos los elementos en un unico valor (suma, promedio).

```
TECNICA soloElite(lista) {
  KI elite = []
  RASTREAR p EN lista {
    SENSAR p >= 8000 {
      elite = ABSORBER(elite, p)
    }
  }
  TRANSMITIR elite
}
```

Leccion 17 - El Tablero (recorridos con un cabezal)

Hasta aca los recorridos fueron sobre listas y rangos. Ahora sumamos una herramienta visual inspirada en Gobstones: **el Tablero**.

El tablero es una grilla de celdas. En cada celda se pueden guardar **esferas del dragon** de cuatro tipos. Hay un **cabezal** llamado `GUERRERO` que se para sobre una celda y se puede mover en cuatro direcciones.

Direcciones: NORTE , SUR , ESTE , OESTE .

Tipos de esfera: ESFERA_1 , ESFERA_2 , ESFERA_3 , ESFERA_4 .

Comandos del tablero:

Comando	Que hace
INICIAR_TABLERO(ancho, alto)	crea un tablero vacio y para al GUERRERO en (0,0)
VOLAR(direccion)	mueve el cabezal una celda en esa direccion
PUEDE_VOLAR(direccion)	dice si se puede mover (CANON/RELLENO) sin caerse del borde
CARGAR(esfera)	pone una esfera en la celda actual
DRENAR(esfera)	saca una esfera de la celda actual
HAY(esfera)	dice si hay al menos una esfera de ese tipo
CUANTAS(esfera)	devuelve cuantas esferas de ese tipo hay
POSICION_X() / POSICION_Y()	columna y fila donde esta el cabezal
MOSTRAR_TABLERO()	dibuja el tablero en pantalla

Ejemplo minimo:

```
INICIAR_TABLERO(3, 3)
CARGAR(ESFERA_1)
CARGAR(ESFERA_1)
VOLAR(NORTE)
CARGAR(ESFERA_2)
MOSTRAR_TABLERO()
SCOUTER("Esferas 1 abajo: " + CUANTAS(ESFERA_1))
```

El origen (0,0) esta **abajo a la izquierda**. NORTE sube, SUR baja, ESTE va a la derecha y OESTE a la izquierda (igual que Gobstones).

Leccion 18 - Recorrido completo del tablero

Combinamos el cabezal con los patrones de recorrido: visitamos **todas** las celdas de la grilla y hacemos algo en cada una (por ejemplo, contar o cargar esferas). El truco es recorrer fila por fila usando PUEDE_VOLAR para saber cuando llegamos al borde.


```
TECNICA recorrerFila(ancho) {  
  GRAVEDAD ancho {  
    CARGAR(ESFERA_1)  
    SENSAR PUEDE_VOLAR(ESTE) {  
      VOLAR(ESTE)  
    }  
  }  
}
```

Con este patron podes barrer un tablero de 4x3 y cargar las 12 celdas, contando cuantas esferas colocaste en total. Es el mismo recorrido enumerativo de las listas, pero ahora sobre una grilla en dos dimensiones.

Diccionario completo de palabras clave

Palabra	Significa
KI	declarar una variable
SCOUTER	mostrar en pantalla
SENSAR / ESQUIVAR	si / si no (condicional)
ENTRENAR	mientras (repeticion condicional)
GRAVEDAD	repetir N veces (repeticion simple)
RASTREAR ... EN	recorrer cada elemento
TECNICA	definir funcion o procedimiento
TRANSMITIR	devolver un valor (return)
CAPSULA	definir un registro/clase
ACTIVAR	crear un registro/objeto
EVOLUCIONA	heredar de otro guerrero
LEGENDARIO	miembro estatico (compartido)
YO	el propio registro (self)
FUSION / DESEO / INVERTIR	y / o / no
CANON / RELLENO / VACIO	verdadero / falso / nulo
GUERRERO	el cabezal del tablero
NORTE / SUR / ESTE / OESTE	direcciones del tablero
ESFERA_1 ... ESFERA_4	los cuatro tipos de esfera
INICIAR_TABLERO	crear un tablero
VOLAR / PUEDE_VOLAR	mover el cabezal / consultar si se puede
CARGAR / DRENAR	poner / sacar una esfera
HAY / CUANTAS	consultar si hay / cuantas esferas
POSICION_X / POSICION_Y	columna / fila del cabezal
MOSTRAR_TABLERO	dibujar el tablero en pantalla

Que empiece el entrenamiento!